

Team: Keyboard Gremlins

COS301 Capstone Project
University of Pretoria

Hackathon Platform



SRS

Name	Student Number
Ayush Sanjith	23535424
Jasveer Ramdhaw	23537036
Rene Reddy	23558572
Heinrich Römer	23538181
Kwaku Asiedu (Sam)	18100156

Team Contact: keyboardgremlins@gmail.com

Introduction.....	3
User Characteristics.....	4
Admin (Event Organiser).....	4
Participant (Competitor).....	4
User Stories.....	5
Use Case Diagrams.....	9
Domain Model.....	11
Functional Requirements.....	12
Authentication & Authorisation.....	12
Event Management.....	12
Level & Resource Management.....	13
Solver Management.....	13
Team Management.....	13
Submission Management.....	14
Scoring Engine.....	14
Leaderboard & Timer.....	14
Live Operations & Dashboard.....	15
Architectural Requirements.....	16
Architecture Diagram:.....	16
Architecture Patterns.....	17
Quality Requirements.....	18
Technology Requirements.....	19
Frontend.....	19
Backend.....	19
Database:.....	19

Introduction

Running a hackathon involves much more than simply hosting a competition. Organizers need a platform that handles the complete event lifecycle, from initial setup and team registration through to submission scoring, live leaderboards and event insights. Without a purpose built system, each hackathon becomes an operationally heavy event that is difficult to repeat or upscale.

The Hackathon Platform is a cloud agnostic, full-stack solution that is purpose built for deterministic optimization challenges. Participants generate an output from provided inputs, rules, constraints and levels and include a solver that checks the output to produce a score. The team/contestant with the highest score wins. The platform allows organizers to create and configure events which allow participants to register, form teams, submit solutions, receive scoring feedback and logs and track their progress using live leaderboards.

User Characteristics

Admin (Event Organiser)

- **Background:** An Admin is likely a developer, technical lead, or recruiter with a strong understanding of the hackathon's technical requirements and deterministic optimization challenges.
- **Goals:** Admins want to run a smooth, repeatable hackathon with minimal manual overhead and standardized core lifecycles.
- **Technical Tasks:** Admins want to use the admin portal to manage configurations, monitor platforms health, and handle file uploads (this can be either event setup or solvers).

Participant (Competitor)

- **Background:** Participants are generally software developers, students or community members with problem-solving skills
- **Goals:** Participants need to solve the provided problem, maximize their scores which is reflected on a live leaderboard and receive real-time feedback on their submissions
- **Technical Tasks:** Participants need to be comfortable in navigating the player portal which will allow them to download resource files, manage teams (this can be registering a team, leaving a team or joining a team) and they must also be able to upload source code archives and output files.

User Stories

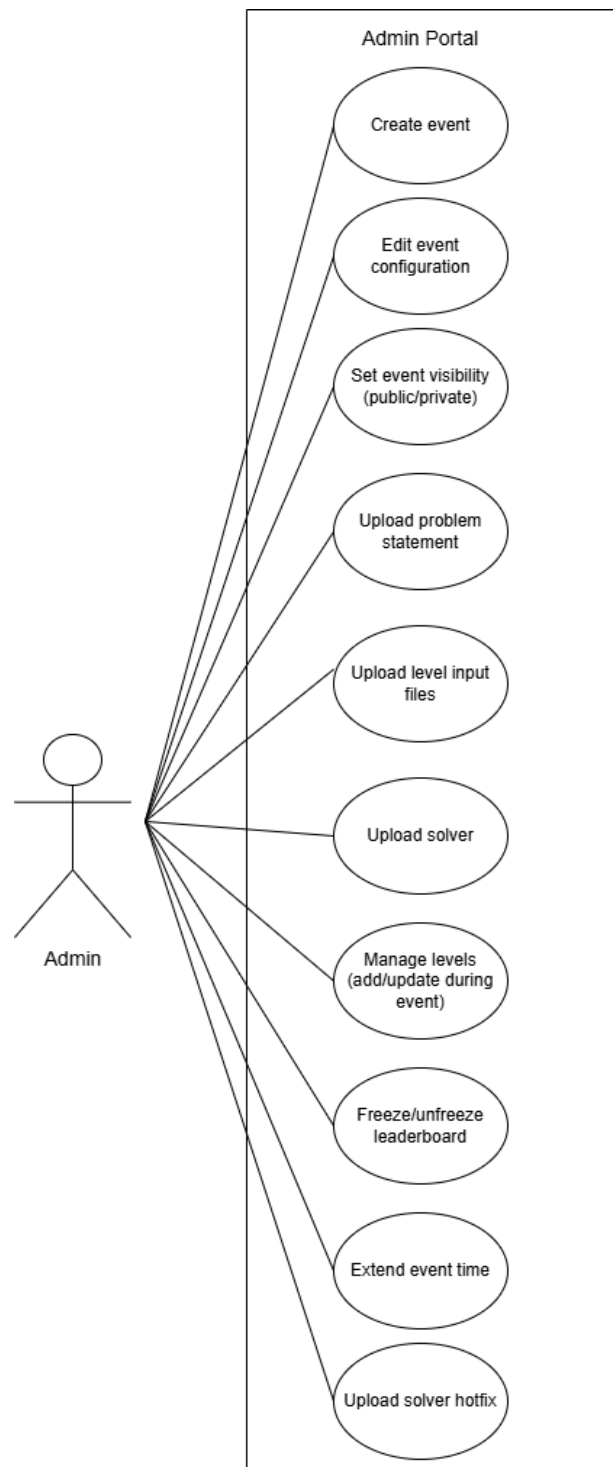
Admin (Event Organiser)	
User Stories	Acceptance Criteria
As an Admin , I want to log in using JWT-based authentication , So that my session remains secure and my access is strictly restricted to administrative functions .	Given an admin provides valid login credentials on the login page, When the login request is processed, Then the system returns a secure JWT and grants access to the Admin Portal.
As an Admin , I want to create and edit event configurations (this includes name, start time, duration and visibility) , So that I can define the operational parameters of a specific hackathon.	Given the admin is authenticated and is on the event setup page, When they enter the event metadata and save, Then the event is successfully initialized with the specific rules and limits (team limit and time limit).
As an Admin , I want to manage operations such as freezing the leaderboard or extending the time , So that I can maintain control over the event in real time.	Given A event is currently active, When the Admin triggers a time extension or leaderboard freeze, Then the system immediately updates the participant's view and stops score updates accordingly.
As an Admin , I want to be able to upload the official solver , So that the platform can use it to grade participants output and produce deterministic scores .	Given the admin is on the event configuration page, When they select and upload a valid solver file,

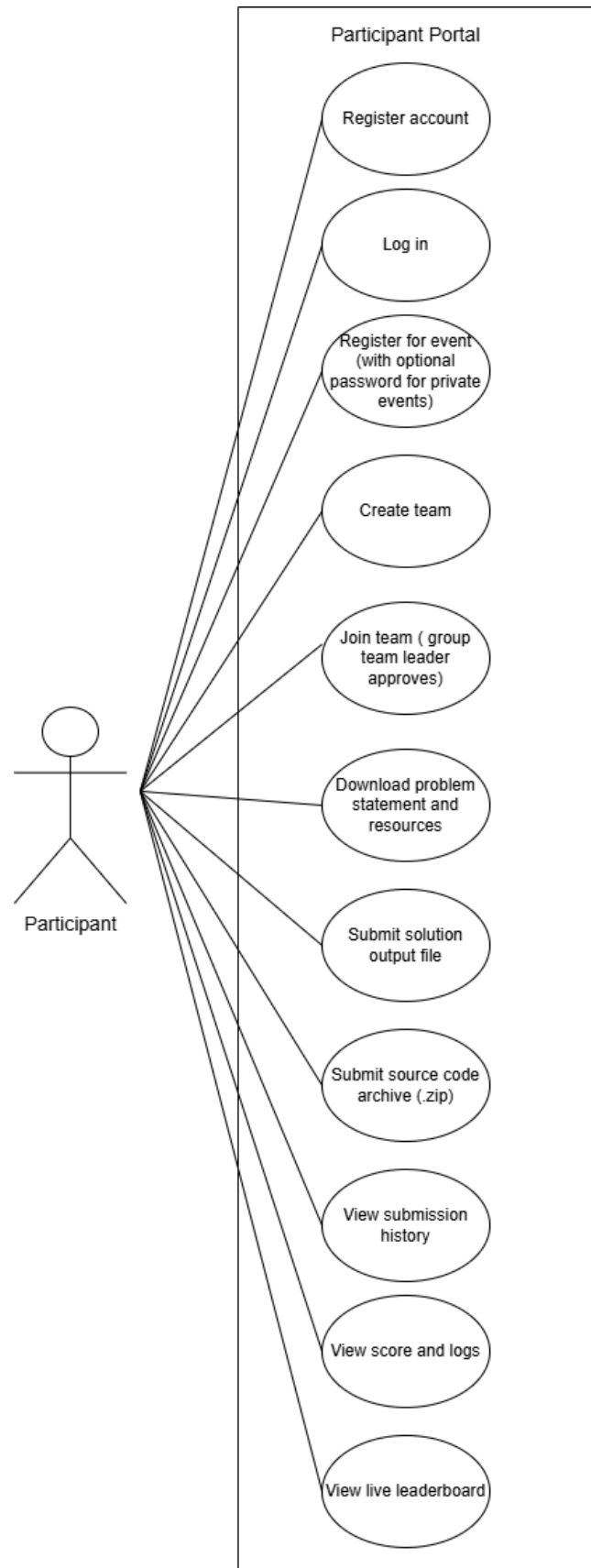
	Then the file is stored in object storage and associated with the current hackathon for scoring.
As an Admin , I want to upload solver fixes and manage solver versions , So that I can address bugs in the scoring logic and apply hot fixes when needed without disrupting the entire events lifecycle .	Given there is already a solver associated with an event, When the Admin uploads a fixed version of the solver script, Then the system updates the scoring logic and re-scores previous submissions for fairness.
As an Admin , I want to view real-time operational information , So that I can ensure that the system handles submission loads effectively .	Given an event is currently active, When the Admin accesses the Coordinator Dashboard , Then the system displays live statistics, this includes submissions per minute, active teams and solver runtime trends.
Participants (Competitor)	
As a Participant , I want to log in using JWT-based authentication , So that my account is protected and I can securely access the player portal.	Given a Participant provides valid login credentials on the login page, When the login request is processed, Then the system returns a secure JWT and grants access to the participant such that they can access their portal.
As a Participant , I want to register for a specific hackathon event , So that I am eligible to participate and can access the events material when it begins.	Given the Participant has logged in and view all the events, When they select “register” and provide a password if required, Then the system associates the user with that event and enables team management features for that event.

As a Participant, I want to create a team for an event, So that I can act as a team lead and invite other members to join me.	Given the Participant is registered for an event but not yet a part of a team, When they choose “Create Team” option and provide a valid team name, Then the team is created, and the Participant is assigned as the primary member.
As a Participant, I want to request to join an existing team, So that I can collaborate with other competitors in a shared team structure.	Given an existing team has space open according to the events rules, When a Participant submits a join request, Then the request is sent through the approval flow for the team lead to accept or decline.
As a Participant, I want to I want to download problem statements, rules and the resource files of an event, So that I have all the necessary inputs to develop my solution offline.	Given the hackathon event has officially started, When the participant clicks the download link for the specific problem spec or level bundles, Then the files are retrieved from object storage and downloaded to the user’s device.
As a Participant, I want to upload a solution output file and a zipped source code archive, So that my output can be scored and my code is archived for later verification.	Given the Participant has a solution for a specific level, When they upload both the output artifact and the .zip source code archive, Then the platform stores both files in object storage and queues the output for the official solver to grade.

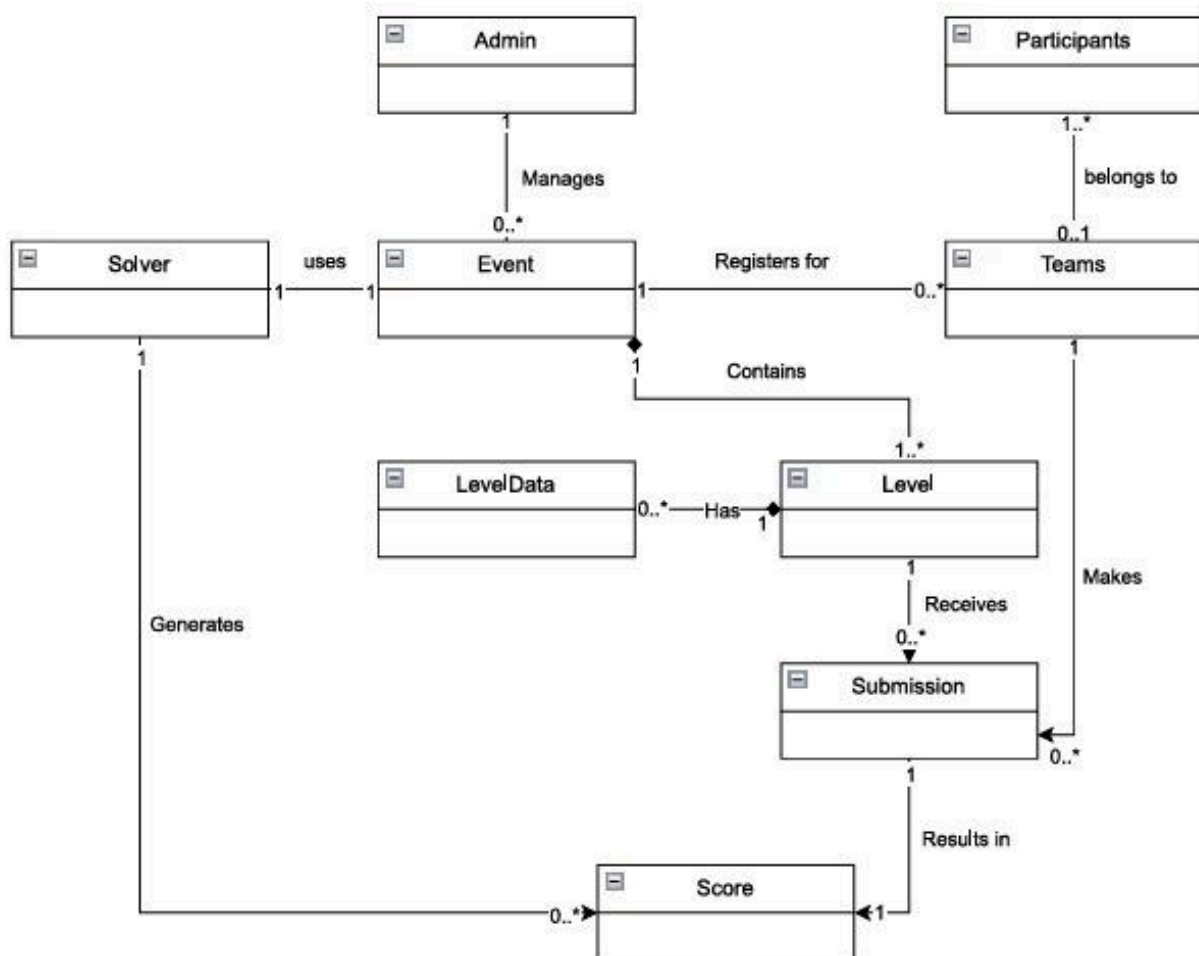
As a Participant, I want to view my submission history, scoring logs and view the live leaderboard, So that I can understand why my submission failed or see how my current score compares to other teams.	Given a submission has been processed by the solver, When a Participant views the level details or event leaderboard, Then the system displays the numerical score, success or error status, detailed feedback logs and the teams current rank on the leaderboard.
--	---

Use Case Diagrams





Domain Model



Functional Requirements

Authentication & Authorisation

FR-1: A user shall be able to register using an email address and a password.

FR-2: A user shall be able to log in and be authenticated with a JWT Token.

FR-3: An Admin shall be able to assign a user as an Admin or Participant for a specific event.

FR-3.1: A user may be an Admin for one event and a Participant in another event.

FR-4: A user shall be able to register for a private event using a registration password/key.

FR-4.1: An incorrect or missing key shall prevent registration and return a descriptive error message.

Event Management

FR-5: An Admin shall be able to create a new hackathon event specifying: name, start date/time, duration, maximum team size, visibility (public/private), rules, prizes, and branding assets.

FR-5.1: An Admin shall be able to edit any event configuration field at any time before or during the event.

FR-6: An Admin shall be able to set a leaderboard freeze time for an event, after which the public leaderboard no longer updates.

FR-6.1: The freeze time shall be configurable by the Admin as an absolute timestamp or as a duration before the event end.

FR-7: An Admin shall be able to upload branding assets (logo, banner images) for an event.

FR-8: An Admin shall be able to extend the event end time during a live event without interrupting ongoing submissions.

Level & Resource Management

FR-9: An Admin shall be able to create levels for an event dynamically at any point during the event lifecycle; levels are not a fixed static set.

FR-10: An Admin shall be able to upload one or more input/resource files per level.

FR-10.1: An Admin shall be able to update or replace level files during a live event.

FR-11: Participants registered for an event shall be able to view and download all level resource files once the event has started.

Solver Management

FR-12: An Admin shall be able to upload a solver file used to score participant submissions for a specific event.

FR-13: An Admin shall be able to upload a hotfix solver version to replace the active solver during a live event.

FR-14: An Admin shall be able to trigger rescoring of existing submissions when a solver is replaced.

Team Management

FR-15: A Participant shall be able to create a team for a specific event.

FR-16: A Participant shall be able to request to join an existing team for a specific event.

FR-16.1: Team join requests shall follow an approval flow: the request is pending until a current team member approves or rejects it.

FR-17: An Admin shall be able to view, manage, and modify teams and their members for any event.

Submission Management

FR-18: For each level, a Participant (via their team) shall be able to upload a solution output file — the artifact that the solver will grade.

FR-19: Each submission shall require an accompanying zipped source code archive (.zip) from the User, containing the implementation that produced the output.

FR-20: An Admin shall be able to download submitted source code archives for auditing.

Scoring Engine

FR-21 A Participant shall be able to see the score, status, and logs for their submission after processing completes.

Leaderboard & Timer

FR-22: A Participant shall be able to view a live leaderboard per event showing team rankings.

FR-23: An Admin shall be able to freeze the leaderboard at a configured time, after which the public view no longer reflects new scores.

FR-23.1: An Admin shall be able to manually freeze or unfreeze the leaderboard at any point during a live event.

FR-24: Participants shall see a countdown timer showing time remaining in the event and time until the leaderboard freezes.

Live Operations & Dashboard

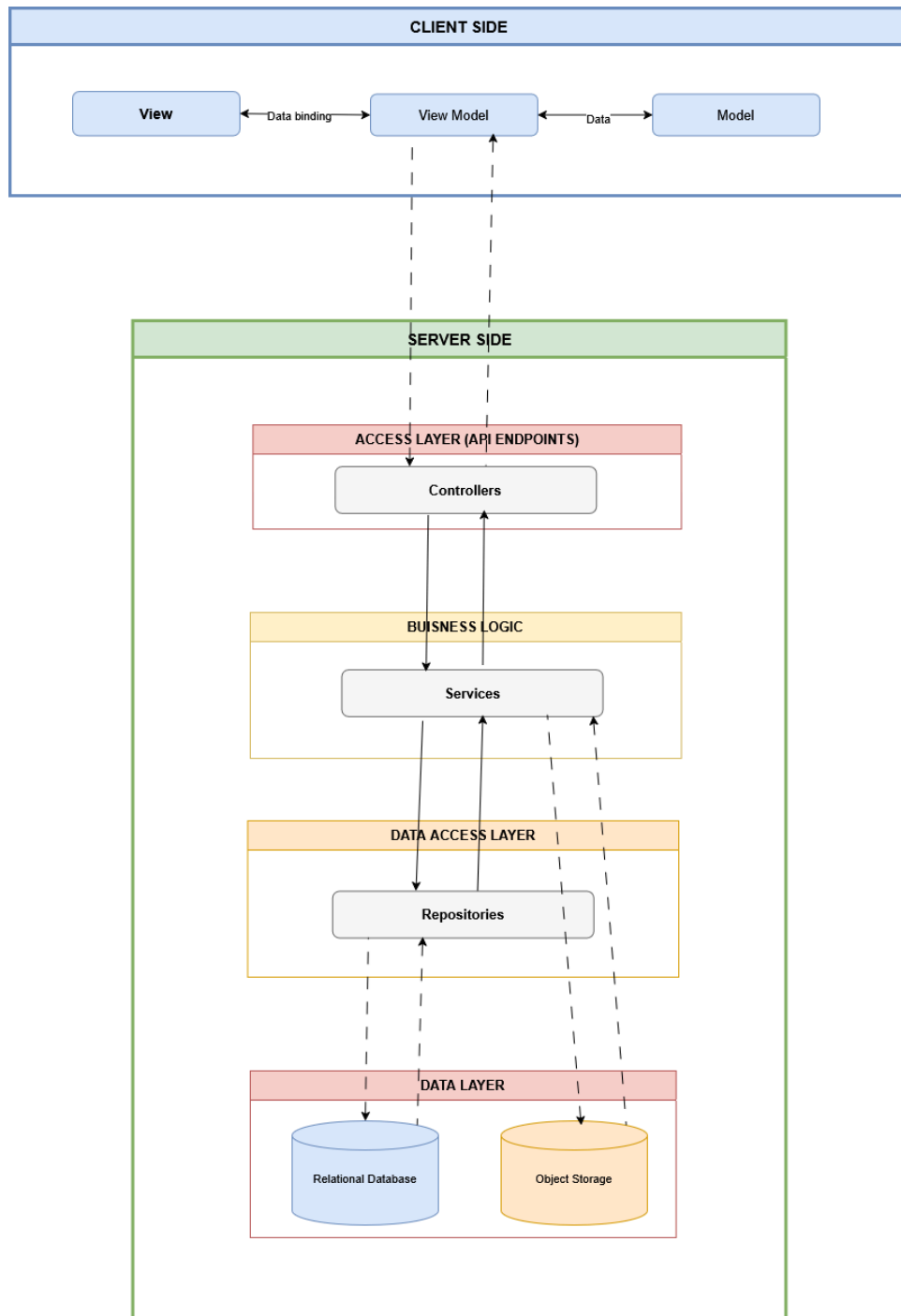
FR-25: An Admin shall be able to view live event statistics including: submissions per minute, number of active teams, score distributions, error rates, and common validation failures.

FR-26: An Admin shall be able to view platform health indicators (e.g. queue depth, worker availability, storage usage).

FR-27: An Admin shall be able to update the problem statement and resource files during a live event without restarting the event.

Architectural Requirements

Architecture Diagram:



Architecture Patterns

At the highest level, the system follows a Client Server architectural style. The Angular Single Page Application acts as the client, running in the user's browser. The Spring Boot REST API acts as the server. The client and server communicate exclusively over HTTP using JSON formatted REST requests. The client never accesses the database or object storage directly, all data access is mediated by the server.

Layered architecture (Server)

The server follows a 4 tier layered architecture that is separated into distinct layers that only communicate downward.

- Controller Layer - Receives HTTP requests and returns responses. This is handled by the Spring MVC RestController.
- Service Layer - This contains the logic and validation of the system. This is handled by the Spring Service.
- Data Access Layer - This layer queries the database and persists to the database. This is handled by the Spring Data JPA Repositories.
- Data Layer - This is where the data is physically stored. This consists of the PostgreSQL DB and the Azure Blob storage.

MVVM (Angular client side)

- Model - This consists of the typescript interface and service components.
- View - This consists of the Angular HTML templates which displays data and emits user events. There is no logic in these components.
- ViewModel - This consists of Angular component classes, these call services.

Quality Requirements

1. Performance:

- Initial load - The web application load completes within 3 seconds.
- Lighthouse performance - Minimum score of 90 on audit tests are required for each page.
- API Response time - API Requests should respond within 500ms.

2. Scalability:

- Peak submission spike - Submission should not degrade under 200 concurrent submissions.
- Worker scaling - Scoring workers should scale and additional worker instances can be added.

3. Reliability:

- Submissions - A submission stored in the object storage is never lost.
- Data integrity - All database writes use transactions where it can rollback writes if there are failures.

Technology Requirements

Frontend

- Angular 17 - Frontend framework
- TypeScript 5 - Frontend functionality
- Angular Material - UI component library
- RxJS - HTTP and WebSockets

Backend

- Java 21 - Backend language
- Spring boot 3.2 - REST API framework
- Spring Security 6 - Authentication and authorization
- Spring Data JPA - ORM
- Flyway 10 - Auto database migrations
- JUnit 5 - Unit and integration tests
- JaCoCo - Code coverage
- Maven 3.9 - Build Java system and dependency management
- Docker - containerization

Database:

- PostgreSQL 16 - RDBMS
- Azure Blob storage - Object storage